

Simuler pour animer

Eric Maisel

Printemps 2026

1 Introduction

Un monde virtuel n'est pas qu'une simple maquette constituée d'objets fixes et qu'on peut explorer. La crédibilité de ce monde est d'autant plus importante qu'il contient des objets semblant avoir des comportements qu'on peut inférer à partir de leur comportement.

Souvent les animations de ces objets s'effectuent par interpolation de valeurs clés (nombres, vecteurs, couleurs, ...). Mais cela reste limité par rapport à la réalité. Nous nous basons pour cela sur une citation du psychanalyste Jacques Lacan selon lequel "le réel c'est quand on se cogne". Pour de nombreux exégètes il s'agit de parler des limites du langage à décrire le monde réel. Ici nous détournons la citation et l'interprétation - peut-être à tort - ainsi : les mondes virtuels sont d'autant plus crédibles qu'ils contiennent des objets autonomes qui n'agissent pas toujours dans le sens souhaité par l'utilisateur mais qui s'adaptent à leur environnement dynamique de façon à atteindre un objectif qui lui est assigné.

Pour atteindre cet objectif on s'inspire des lois de la Nature pour pouvoir prendre en compte l'imprévisibilité provoquée par les interactions entre de nombreux objets dont des utilisateurs humains.

Il ne s'agit plus d'animer un monde virtuel mais de le simuler : à partir d'une situation à un instant t et de règles (les lois du monde virtuel à créer) on cherche à calculer l'état de ce monde virtuel aux instants suivants.

On s'intéresse ici à une architecture destinée à faciliter la mise en oeuvre de tels simulateurs.

2 Un cube qui tourne sur lui-même

On veut créer une scène dans laquelle un cube tourne sur lui-même. On modifie pour cela la classe `World` et certaines des méthodes de cette classe :

- `createWorld` pour créer le cube
- `update` pour faire tourner le cube

```
class World extends Simu {  
  ...  
  createWorld(){  
    ..  
    const cube = PRIMS.boite("cube", {}, this.monde.scene) ;  
    cube.position.y = 2.0 ;  
    ...  
  }  
  
  update(dt){  
    const dt = this.engine.getDeltaTime()/1000.0 ;  
    this.horloge += dt ;  
  
    const mesh = this.scene.getMeshById("cube") ;  
    if(mesh != null){  
      const angle = (this.horloge % 360)*Math.PI/360.0 ;  
      mesh.rotation.set(angle, 2*angle,0) ;  
    }  
  }  
}
```

Cela pose un problème : modifier le monde virtuel impose de modifier le code du simulateur. De façon générale il est préférable d'éviter de modifier un code qui fonctionne ... Il faut trouver une autre architecture logicielle plus décentralisée et séparer les parties propres au simulateur de celles qui concernent les objets simulés.

3 Des cubes qui tournent sur eux mêmes

Dans l'exemple précédent modifier le comportement d'un objet animable oblige à modifier le code du simulateur. Pour éviter cela on introduit une nouvelle notion : celle d'entité. Une entité représente une "chose" du monde virtuel

- qu'il est possible de distinguer des autres (ici par un identifiant unique)
- qui est incarnée : on peut la voir, l'entendre, la toucher
- qui est dotée d'un état
- à laquelle est associée une procédure (`update(dt)`) qui fait évoluer cet état.

Ces objets sont implémentés par la classe `Entity`. Un des intérêts de cette approche est de pouvoir placer la logique de simulation non pas dans la méthode `update` du simulateur mais dans la méthode `update` de la sous-classe de `Entity` qui modélise cet objet.

La méthode `update` du simulateur, exécutée régulièrement, aura comme responsabilité d'invoquer les méthodes `update` de chaque entité du monde virtuel.

3.1 Définition des entités

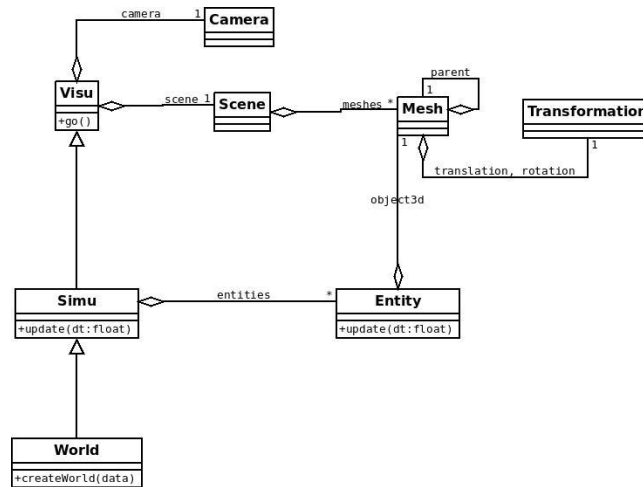


FIGURE 1 – Modèle basé entités

```

class Entity {
  constructor(id, data, world){
    this.id = id ;
    this.world = world ;
    this.object3d = data.object3d || null ;
    this.position = data.position||new BABYLON.Vector3(0,0,0);
  }

  update(dt){}
}

```

La classe `Simu` gère une collection d'entités. Elle possède les attributs et les méthodes qui permettent d'ajouter, de supprimer des acteurs mais aussi de faire évoluer leur état.

```

class Simu extends Visu {
  constructor(){
    this.entities = [] ;
  }

  createEntity(nom, Type, data){
    const entity = new Type(name, data, this) ;
    this.directory[name] = entity ;
    this.entities.push(entity) ;
    return entity ;
  }

  ...

  update(dt){
    this.entities.forEach((e)=>{e.update(dt);}) ;
  }
}

```

Le fichier `entities.js` contient différents types d'entités : par exemple les entités "Alea" qui se déplacent de façon aléatoire ou les entités qui tournent sur elles-mêmes. Ces différents types d'entités sont définies en créant des sous-classes de la classe `Entity`.

Voici la définition d'une classe d'entités qui se déplacent de façon aléatoire :

```

class Alea extends Entity {

  constructor(id, data, monde){
    super(id, data, monde) ;
    this.object3d = data.object3d ||
      PRIMS.box(id, {}, monde.scene) ;
    this.vitesse = data.vitesse || 1.0 ;
    this.vit      = new BABYLON.Vector(0,0,0) ;
  }

  update(dt){
    const vx = (0.5 - Math.random())*this.vitesse ;
    const vz = (0.5 - Math.random())*this.vitesse ;
    this.vit.set(vx,0, vz) ;
    this.vit.scaleAndAddToRef(dt, this.position) ;

    this.object3d.position.copyFrom(this.position) ;
  }
}

```

De la même façon une classe d'entités qui tournent sur elles-mêmes :

```

class Rotateur extends Entity {

  constructor(id, data, monde){
    super(id, data, monde) ;
    this.angle = 0.0 ;
    this.object3d = data.object3d ||
      PRIMS.box(id, {}, monde.scene);
  }

  update(dt){
    this.angle += Math.PI/180.0 ;
    this.object3d.rotation(0, this.angle, 0) ;
  }
}

```

Ces deux classes d'entités sont exportées du fichier `entities.js` de la façon suivante :

```
const ENTITIES = {
  alea : Alea,
  rot  : Rotateur
} ;

export {ENTITIES} ;
```

3.2 Utilisation des entités

```
import {ENTITIES} from './entities.js' ;

class World extends Simu {

  constructor(){ ... }

  createWorld(){
    this.createEntity("e1", ENTITIES.alea, {}) ;
    this.createEntity("e2", ENTITIES.alea, {}) ;
    this.createEntity("e3", ENTITIES.rot,  {}) ;
  }

}
```

3.3 Exercice

Exercice

On veut simuler une pendule. Pour l'instant on se contente d'un simple pendule constitué d'une tige de longueur 1 avec une masse cylindrique à son extrémité la plus basse de rayon r . Ce pendule oscille autour d'un axe parallèle à l'axe du cylindre avec une période de T seconde(s).

Indice : la partie graphique (l'incarnation) de la pendule sera ici créée dans le constructeur de la classe **Pendule**, tandis que les instructions de contrôle de son mouvement le seront dans sa méthode **update**.

Exercice optionnel

Proposez une classe **Horloge** qui permette non seulement la création d'une horloge (avec au moins un pendule tel que défini ci-dessus mais également une aiguille des heures et une aiguille des minutes) mais également une animation **en temps réel** : les aiguilles indiquent une heure dans le monde virtuel correspondant au temps virtuel qui s'écoule à la même vitesse que le temps physique.

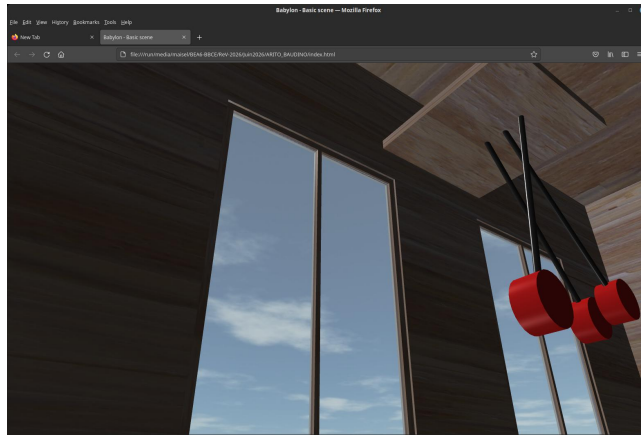


FIGURE 2 – Un tri-pendule

3.4 Problème

On peut décrire ainsi de nombreux types d'entités. Mais on se heurte vite à un problème : celui de l'explosion combinatoire. Prenons un exemple simple : comment décrire un cube qui a un mouvement propre de rotation tout en se déplaçant de façon aléatoire (pensez à la Terre qui tourne sur elle-même tout en orbitant autour du soleil). L'architecture impose ici de décrire une nouvelle classe qu'on pourrait appeler **AleaEtRot**. Au final cela fait exploser le nombre de classes et la taille du code produit.

On peut certes utiliser l'héritage multiple pour remédier à ce problème mais outre le fait que tous les langages ne le permettent pas, cela pose des problèmes bien connus en génie logiciel et ne réduit pas le nombre de classes à coder.

4 Entités et composants

La solution retenue pour surmonter le problème évoqué ci-dessus s’inspire de l’architecture Entité-Composant-Système. Chaque entité est représentée par une instance d’une sous-classe de la classe **Entity**. Une entité n’est caractérisée que par un identifiant et une liste d’instances de sous-classes de la classe **Component**. Ce sont ces composants qui définissent l’aspect et le comportement des entités auxquelles ils sont associés. Certains de ces composants ne seront exécutés qu’une seule fois (par exemple ceux qui donnent un aspect ou un positionnement initial) tandis que d’autres le seront régulièrement.

On peut alors remarquer que :

- le code qui décrit l’aspect et le comportement des objets n’est plus dans le simulateur
- le code qui décrit l’aspect et le comportement des objets n’est pas dans les classes qui définissent des entités mais dans les composants associés à ces objets.

L’intérêt de cette approche réside dans sa flexibilité et la possibilité d’ajouter et/ou supprimer des composants dynamiquement aussi bien à la conception de la simulation (pour faciliter le prototypage) que lors de son exploitation (pour que les entités puissent s’adapter à l’évolution de leur environnement).

Cette nouvelle architecture peut être représenté par le diagramme de classes suivant :

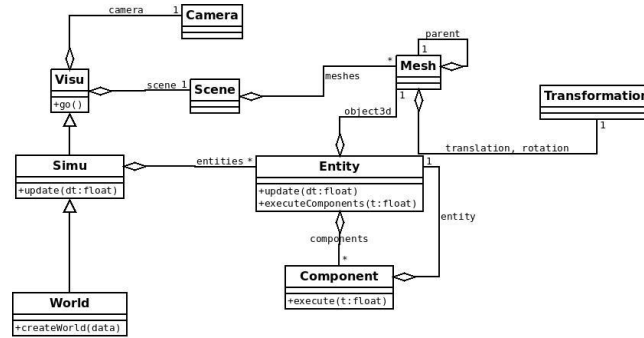


FIGURE 3 – Modèle Entités-Composants

La création des entités s’effectue dans la méthode `createWorld` de la classe `World` :


```

import {Simu}      from './simu.js' ;
import {ENTITIES} from './entities.js' ;

class World extends Simu {
  ...

  createWorld(){
    ...
    this.createEntity("e0", ENTITIES.Entity,{}) ;
    ...
  }
}

```

On veut créer un monde virtuel dans lequel une boîte est placée au point de coordonnées (3,0,5) et est orientée selon une rotation d'axe vertical et d'angle 45°.

Comme le montre le code suivant la création de l'incarnation de l'entité "e0" se fait en utilisant le composant `boite`. Son placement (position, orientation) dans l'espace du monde virtuel se fait au moyen des composants `position` et `rotation`. Ces composants sont définis dans le fichier `component.js`.

Remarquez dans ce code que les méthodes `createEntity` et `add` renvoient l'instance de l'entité qui est en train d'être construite.

```

import {Simu}      from './simu.js' ;
import {ENTITIES} from './entities.js' ;
import {COMPS}     from './components.js' ;

class World extends Simu {
  ...

  createWorld(){
    ...
    this.createEntity("e0", ENTITIES.Entity,{}) .
    add(COMPS.box, {width:5, height:3, depth:0.05 }) .
    add(COMPS.position,{x:3,z:5}) .
    add(COMPS.rotation,{y:Math.PI/4.0}) ;
    ...
  }
}

```

4.1 Classes Component et Entity

La classe **Component** peut être vue comme une classe abstraite de laquelle seront dérivés des composants plus spécifiques.

Le composant connaît l'entité auquel il est associé (attribut **entity**)

Les rôles des différentes méthodes sont les suivants :

- **constructor** : instancie un composant en le paramétrant par des données qui lui sont spécifiques et surtout par l'entité sur laquelle le composant va pouvoir agir
- **register** : associe le composant en train d'être créé à la liste des composants dont le code doit être exécuté régulièrement
- **execute** : code qui permet de modifier certaines variables du composant ou de l'entité à laquelle le composant est associé.

```
class Component {
  constructor(data, entity){
    this.entity = entity ;
  }

  register(){
    this.entity.components.push(this) ;
  }

  execute(){}
```

On redéfinit la classe **Entity** de façon à pouvoir gérer une collection de composants. Ceci se fait en ajoutant :

- l'attribut **components** qui associe à l'entité les composants dont il faudra régulièrement invoquer la méthode **execute**
- la méthode **add** dont le rôle est de créer de nouveaux composants destinés à agir sur cette entité
- la méthode **execute** dont le rôle est d'invoquer la méthode **execute** des composants référencés par l'attribut **components**

Pour ne pas compliquer la compréhension de l'architecture logicielle les méthodes de recherche et de suppression de composants ne seront pas présentées ici.

```

class Entity {
  constructor(id, data, world){
    this.id      = id ;
    this.world    = world ;
    this.position = data.position || new BABYLON.Zero() ;
    this.object3d = data.object3d || null ;
    this.components = [] ;
  }

  add(componentType, data){
    const comp = new componentType(data, this) ;
    return comp ;
  }

  getComponent(ComponentType) {
    return this.components.find(c =>
                                     c instanceof ComponentType);
  }

  execute(){
    this.components.forEach((c)=>{c.execute();}) ;
  }

  update(dt){}
}

```

L'utilisation de composants nécessite une modification de la méthode `update` de la classe `Simu`.

```

class Simu extends Visu {
  constructor(){ ... }

  update(dt){
    // Exécution du code des composants des entités
    this.entities.forEach((e) => {e.execute();}) ;

    // Mise à jour des entités
    this.entities.forEach((e) => {e.update(dt)})) ;
  }
}

```

4.2 Des exemples

Certains des composants peuvent être considérés comme redondants. Par exemple le composant `CompBoite` qui n'est qu'une encapsulation de la primitive `box`. Mais cela rend homogène la création des composants en ne considérant que deux types d'objets : les entités et les composants.

Composant définissant une incarnation

```
class Box extends Component {
  constructor(data, entity){
    super(data, entity) ;
    const boite = PRIMS.box(entity.id,
                             data,
                             entity.world.scene) ;

    entity.object3d = boite ;
  }
}
```

Composant définissant une transformation géométrique

```
class Position extends Component {
  constructor(data, entity){
    super(data, entity) ;
    const tx = data.x || 0.0 ;
    const ty = data.y || 0.0 ;
    const tz = data.z || 0.0 ;

    entity.position.set(tx, ty, tz) ;
    if(entity.object3d != null)
      entity.object3d.position.set(tx, ty, tz) ;
  }
}
```

Composant définissant un comportement

```
class Rotateur extends Component {
  constructor(data, entity){
    super(data, acteur) ;
    this.register() ;
    this.alpha = data.alpha || Math.PI/180 ;
  }

  execute(){
    if(this.entity.object3d !== null)
      this.entity.object3d.rotation.y += this.alpha ;
  }
}
```

A noter ici la présence dans le constructeur du composant `CompRotateur` de l'instruction `this.register()`. L'exécution de celle-ci permet d'ajouter ce composant dans la liste de l'attribut `components` de l'instance de l'entité à laquelle le composant est associé de façon à ce que la méthode `execute` de ce composant soit invoquée régulièrement.

Export des ressources Component

```
const COMPS = {
  boite    : Box,
  position : Position,
  rotateur : Rotateur,
  ...
} ;

export {COMPS} ;
```

4.3 Exercice(s)

Exercice

1. Proposer les composants **Guide** et **Visiteur** qui permettent de créer les objets graphiques de la figure 4.
2. Placez un guide et 10 visiteurs de façon aléatoire dans le hall du musée.

Attention : il faudra (i) que les dimensions largeurs, hauteurs, diamètres, ...) des personnages soient paramétrables et que (ii) vos modèles 3d résultants ne soient pas qu'une copie de ceux proposés dans les figures ci-dessus.

Indice : bien entendu tout ceci se fait en créant et en utilisant des entités et des composants ...



FIGURE 4 – guide et visiteurs

5 Utiliser des forces pour faire évoluer les mondes virtuels

5.1 Entités cinématiques

Il est possible de contrôler l'état d'une entité en utilisant l'horloge virtuelle du simulateur. Cela permet un contrôle fin mais qui n'est pas intuitif. Une autre façon de faire est de définir le mouvement d'un objet à travers sa vitesse et non plus le temps mais l'intervalle de temps entre les instants qui correspondent au calcul de deux images consécutives :

$$P(t + dt) = P(t) + \vec{v}(t)dt$$

Ou s'il faut calculer la vitesse à partir de l'accélération :

$$\begin{cases} P(t + dt) &= P(t) + \vec{v}(t)dt \\ \vec{v}(t + dt) &= \vec{\gamma}(t)dt \end{cases}$$

Classe Kine : une entité pour la cinématique

Le calcul de l'état de l'entité considérée n'étant plus le même cela implique de définir un nouveau type d'entités, ici en dérivant la classe **Kine** de la classe **Entity**.

Les différences entre les deux classes sont les suivantes :

- deux attributs sont ajoutés : **velocity** et **acceleration**
- la méthode **update** est modifiée de façon à pouvoir intégrer les effets de l'accélération sur la position de l'entité

```
class Kine extends Entity {
    constructor(id, data, wld){
        super(id, data, wld) ;
        this.velocity = BABYLON.Zero() ;
        this.acceleration = BABYLON.Zero() ;
    }

    update(dt){
        this.velocity.scaleAndAddToRef(dt, this.position) ;
        this.acceleration.scaleAndAddToRef(dt, this.velocity) ;

        if(this.object3d){
            this.object3d.position.copyFrom(this.position) ;
        }
    }
}
```

Un exemple de composant pour la cinématique

Un tel composant permet de définir la vitesse d'une instance de la classe Kine.

```
class BrownianMotion extends Component {
  constructor(data, entity){
    super(data, entity) ;
    this.register() ;
    this.p0 = data.p0 || 0.1 ;
    this.vMax = data.vMax || 1.0 ;
  }

  execute(){
    const p = Math.random() ;
    if(this.p0 > p){
      const vx = (0.5-Math.random())*this.vMax ;
      const vz = (0.5-Math.random())*this.vMax ;
      entity.velocity.set(vx,0.0,vz) ;
    }
  }
}
```

Exercice(s)

Exercice

1. Placez dans le hall du musée virtuel qui fait l'objet du projet un guide en utilisant le composant développé plus haut. Faites en sorte qu'il ait un mouvement Brownien (COMPS.brownianMotion dans les ressources disponibles).
2. Faites de même pour les visiteurs
3. Faites en sorte que le guide et les visiteurs regardent dans le sens de leur déplacement. Bien entendu cela va passer par un nouveau composant. Pour cela il faut développer le composant COMPS.lookAtForward. Pour cela vous utiliserez la méthode lookAT(unPoint) des classes Mesh ou TransformNode. Vous pouvez vous inspirer des composants COMPS.lookAtPoint et COMPS.lookAtCamera (fichiers lookAtPoint et lookAtCamera.js dans le répertoire ./js/components).

Attention : il ne s'agit pas de reprendre tel quel le code déjà écrit pour illustrer l'utilisation des personnages guide et visiteurs. Quelques lignes de code devront sans doute être modifiées. Petites modifications mais grands effets!!

Exercice

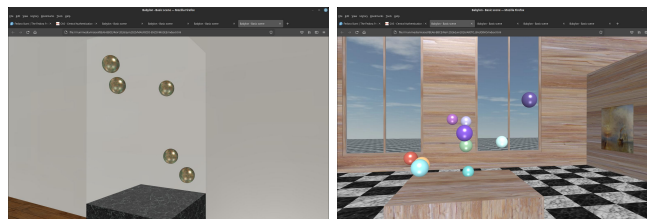
On veut simuler des boules qui se déplacent en ligne droite dans une boîte virtuelle. A chaque fois qu'une balle rencontre une paroi virtuelle elle rebondit dessus. Ainsi si elle rencontre une paroi virtuelle du plan $x = x_0$ sa vitesse va être transformée de la façon suivante : v_y et v_z changent de signe.

1. En utilisant la classe `Kine` et les composants `COMPS.sphere`, `COMPS.position` proposez une implémentation de `Rebond` la sous-classe de `Component` qui gère les rebonds dans une boîte virtuelle de diagonale $P_0(x_0, y_0, z_0), P_1(x_1, y_1, z_1)$ et de cotés perpendiculaires aux axes du repère de la scène du monde virtuel.

Voici un schéma d'utilisation de ce composant (mis en oeuvre dans la méthode `createWorld`) de la classe `World`) :

```
const boule = this.createEntity("boule", ENTITIES.Kine, {})  
    .add(COMPS.sphere, {...})  
    .add(...)  
    .add(COMPS.rebond, {...})  
    .add(...);
```

2. Créez un monde virtuel contenant 10 boules rebondissantes dans une boîte de diagonale $P_0(-1, 0.5, -1), P_1(1, 2, 1)$



Exercice

Faites en sorte que le guide et les visiteurs restent dans le hall. Pour cela réutilisez le composant `Rebond`.

Problème

La difficulté ici est de trouver la valeur de $\vec{v}$ ou de $\vec{\gamma}$ qui permette d'obtenir un déplacement crédible. L'approche cinématique décrite ci-dessus ne permet que difficilement d'obtenir cette crédibilité. Aussi préfère-t-on utiliser une modélisation plus "physique" reposant sur la dynamique Newtonienne.

5.2 La dynamique Newtonienne

La dynamique Newtonienne permet de relier le comportement cinématique d'un objet avec son environnement. Elle repose sur une notion : celle de force. Trois lois décrivent le lien entre ces forces et le comportement cinématique des objets.

Une force est un vecteur qui provoque l'accélération d'un objet doté d'une masse non nulle.

- **Première loi (principe d'inertie)** : tout corps conservera son état de repos ou de mouvement uniforme en ligne droite dans lequel il se trouve, à moins qu'une force ne lui soit appliquée.
- **Seconde loi (principe fondamentale de la dynamique)** : une force résultante exercée sur un objet est toujours égale au produit de la masse de cet objet par son accélération. De plus, l'accélération produite et la force résultante ont la même orientation.

$$\sum \vec{F}_i = m\vec{\gamma}$$

- **Troisième loi (loi de l'action et de la réaction)** : si un corps A exerce une force sur un corps B alors B exerce sur A une force de même intensité, de même direction et de sens opposé.

Seules les deux premières lois vont nous intéresser.

La mise à jour de la position d'un objet repose sur le schéma de double intégration d'Euler :

$$\begin{cases} p(t + dt) &= p(t) + \vec{v}dt \\ \vec{v}(t + dt) &= \vec{v}(t)dt \\ \vec{\gamma}(t) &= \frac{1}{m} \sum \vec{F}_i \end{cases}$$

Le calcul à effectuer pour faire évoluer l'état d'un objet (l'animer) va s'effectuer en trois temps :

- Bilan des forces
- Accumulation des forces
- Calcul de la vitesse et de la position

Ce nouveau schéma d'intégration impose la définition d'une nouvelle classe d'entités : **Newton** qui dérive de la classe **Entity**. Ici l'accélération est remplacée par la force (attribut **force**).

Le bilan des forces est fait par l'association de composants à chaque entité simulée. A chaque pas de temps les forces sont calculées (méthode **execute**) et application des forces (méthode **applyForce**).

```

class Newton extends Entity {

    constructor(id, data, world){
        super(id, data, world) ;
        this.mass      = data.mass || 1.0 ;
        this.velocity = new BABYLON.Vector3(0,0,0) ;
        this.force      = new BABYLON.Vector3(0,0,0) ;
    }

    applyForce(f){
        this.force.addInPlace(f) ;
    }

    update(dt){
        this.velocity.scaleAndAddToRef(dt,this.position) ;
        this.force.scaleAndAddToRef(dt/this.mass, this.velocity) ;
        this.force.set(0,0,0) ;

        if(this.object3d){
            this.object3d.position.copyFrom(this.position) ;
        }
    }
}

```

Les forces qu'on applique sont implémentées par des composants. Ainsi une force horizontale dont les variations de direction et d'intensité sont aléatoires va pouvoir être codée de la façon suivante :

```

class Alea extends Component {

    constructor(data, entity){
        super(data, entity) ;
        this.register() ;
        this.deltaAngle      = data.deltaAngle      || 0.1 ;
        this.deltaIntensity = data.deltaIntensity || 0.1 ;
        this.angle           = 0.0 ;
        this.intensity        = 0.0 ;
        this.force            = new BABYLON.Vector3(0,0,0) ;
    }

    execute(){
        this.angle += (0.5 - Math.random())*this.deltaAngle ;
        this.intensity += (0.5-Math.random())*this.deltaIntensity ;
        this.intensity = Math.max(0,this.intensity) ;
        this.force.set(this.intensity*Math.cos(this.angle),
                        0,
                        this.intensity*Math.sin(this.angle)) ;

        this.entity.applyForce(this.force) ;
    }

}

```

5.3 Exercice(s)

Exercice optionnel

- Programmez un composant `COMPS.impulsion` qui simule une force qui provoque une impulsion sur quelques secondes
- Programmez un composante `COMPS.frottement` qui simule une force de frottement :

$$\vec{F} = -\rho\vec{v}$$

- Testez ces composants sur une entité ayant l'aspect d'un guide. D'abord avec le composant `COMPS.impulsion` puis en ajoutant le comportement `COMPS.frottement`.

6 Contrôle par steering

6.1 Objectif

Donner la possibilité au concepteur d'un monde virtuel de contrôler le comportement d'une entité en leur assignant des objectifs.

6.2 Principe

Autonomie dans un monde virtuel

Le terme "agent autonome" fait généralement référence à des entités qui sont capables de prendre par eux-mêmes les décisions nécessaires pour agir dans un environnement sans dépendre d'un plan global.

Dans les mondes virtuels une entité sera dite autonome si elle possède un objectif qui lui est propre et qu'elle dispose en elle-même des ressources logicielles nécessaires pour réaliser cet objectif malgré les changements non déterminés a priori de cet environnement.

On se limitera ici en terme d'actions aux déplacements dans l'espace mais il devrait être possible d'étendre les résultats obtenus à d'autres types d'actions (modifications d'aspect, mouvements fins, actes de langage, ...).

Ces actions seront réalisées selon le principe Newtonien" vu précédemment : "tout changement d'état cinématique (mouvement) est causé par l'application d'une force". Les entités considérées vont devoir, suite aux décisions prises pour atteindre un objectif, s'appliquer des forces internes (forces motrices). Ceci dans un environnement qui lui également leur applique des forces.

Une entité autonome est une entité logicielle qui satisfait les points suivants :

- un monde virtuel est un ensemble d'entités autonomes. Il s'agit d'un système distribué : il n'y a ni contrôle centralisé ni plan global
- une entité autonome a une perception limitée de son environnement
- une entité autonome possède un objectif (un but) qui lui est propre et les ressources lui permettant le traitement des informations provenant de son environnement et la prise de décision conduisant aux actions nécessaires à la réalisation de cet objectif et ceci même dans un environnement dynamique et non déterminé a priori
- En première approximation on peut considérer qu'une entité autonome suit un cycle "Perception / Décision / Action"

Modélisation

On va considérer trois niveaux de modélisation d'une entité autonome :

- **Incarnation** : elle se fait au moyen d'un objet graphique tridimensionnel
- **Mécanique** : une entité est considérée comme étant une masse ponctuelle orientée
- **Comportementale** : trois niveaux sont considérés

- **Niveau stratégique** qui permet de décider des objectifs à atteindre
- **Niveau tactique** qui permet de sélectionner les actions à effectuer afin d'atteindre ces objectifs
- **Opérationnel** qui réalise ces actions

Fondement du steering

Le steering tel qu'il a été proposé par Reynolds repose sur la résolution d'un problème inverse : il s'agit, pour un objectif donné assigné à une entité, à déterminer la vitesse $\vec{v}_d$ à donner à cette entité pour réaliser cet objectif. On considère que la vitesse courante de l'entité est $\vec{v}_c$. Passer de la vitesse $\vec{v}_c$ à la vitesse $\vec{v}_d$ correspond à faire subir à l'entité une accélération (variation de vitesse instantanée) qui peut être assimilée à l'application d'une force dite de steering (cadre de la mécanique Newtonienne) appelée par la suite $\vec{F}$.

Reynolds, l'initiateur du steering, propose dans les travaux qui ont conduit à son article de 1987 "Flocks, Herds and Schools : A Distributed Behavioral Model" dans la revue "Computer Graphics" la relation suivante :

$$\vec{F}_s = \vec{v}_d - \vec{v}_c$$

Cette force sera appliquée à l'entité contrôlée.

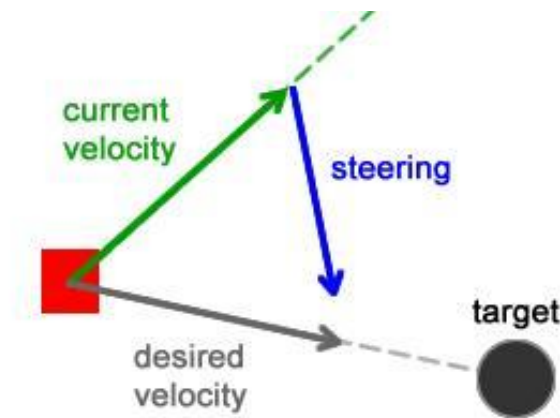


FIGURE 5 – principe du steering

De façon à rester réaliste on supposera que la norme des vitesses et des forces peuvent être limitées (inférieures respectivement à v_M et F_M des valeurs caractéristiques des entités à animer).

Il s'agit donc, étant donné une entité et un objectif :

- de calculer la vitesse $\vec{v}_d$ permettant à l'entité de s'approcher de la réalisation de l'objectif

- d'appliquer la force de steering $\vec{F}_s = k(\vec{v}_d - \vec{v}_c)$ à l'entité à animer

6.3 Exemples

Comportement basique : passer par un point (Seek)

L'exemple de comportement le plus simple est celui qui consiste à faire passer une entité par un point donné C . On appelle ce comportement **Seek**.

L'entité est située au point P , animé d'une vitesse $\vec{v}_c$. Il peut atteindre une vitesse de norme maximale v_M . On cherche à la faire passer par un point cible C .

Le vecteur vitesse désirée $\vec{v}_d$ est colinéaire au vecteur $\vec{PC}$. On peut alors écrire :

$$\vec{v}_d = k\vec{PC}$$

On cherche à faire aller l'entité le plus rapidement de P à C . On peut alors préciser la vitesse désirée :

$$\vec{v}_d = \frac{v_M}{|\vec{PC}|} \vec{PC}$$

La force de steering à appliquer sera donc :

$$\vec{F}_s = \min(|\vec{F}|, \frac{F_M}{|\vec{F}|}) \vec{F} \text{ avec } \vec{F} = \frac{v_M}{|\vec{PC}|} \vec{PC} - \vec{v}_c$$

En ce qui concerne la mise en oeuvre de ce comportement :

- L'entité sera mise en oeuvre par une instance de la classe **Newton**
- Le comportement permettant de passer par un point cible sera quant à lui mis en oeuvre par une classe **Seek** sous-classe de la classe **Component**.

```

class Seek extends Component {
  constructor(data, entity){
    super(data, entity) ;
    this.register() ;
    this.vM    = data.vM      || 1.0 ;
    this.fM    = data.fM      || 1.0 ;
    this.C     = data.posCible || new BABYLON.Vector3(0,0,0) ;
    this.force = new BABYLON.Vector3(0,0,0) ;
    this.vd    = new BABYLON.Vector3(0,0,0) ;
  }

  execute(){
    this.subtractToRef(this.entity.object3d.position, this.vd);
    this.vd.normalize();
    this.vd.scaleInPlace(this.entity.vMax);
    this.vd.subtractToRef(this.entity.velocity, this.force);

    this.entity.applyForce(this.force) ;
  }
}

```

Arrêt en un point (Arrive)

On constate vite que le comportement **Seek** permet de passer par un point mais pas de s'y arrêter. Généralement l'entité contrôlée dépasse le point visé de façon suffisamment importante pour considérer qu'elle l'a raté : elle se retourne sur elle même et y retourne ... S'ensuit un phénomène d'oscillations autour du point cible.

La solution pour s'arrêter en un point cible consiste à définir autour de ce point une zone de freinage dans laquelle l'entité contrôlée va progressivement diminuer sa vitesse. Ce comportement est appelé **Arrive** (cf. figure 6)

On cherche à faire s'arrêter en un point C une entité représentée par un point P et animé d'une vitesse $\vec{v}_c$. On associe au point d'arrêt C une zone de freinage centrée sur C et de rayon d_0 .

Le principe est d'utiliser le comportement **Seek** en faisant varier la vitesse v_M en fonction de la distance de l'entité contrôlée à la cible :

$$v_M = \begin{cases} v_0 & \text{si } d \geq d_0 \\ k \frac{v_0}{d_0} & \text{si } d < d_0 \end{cases}$$

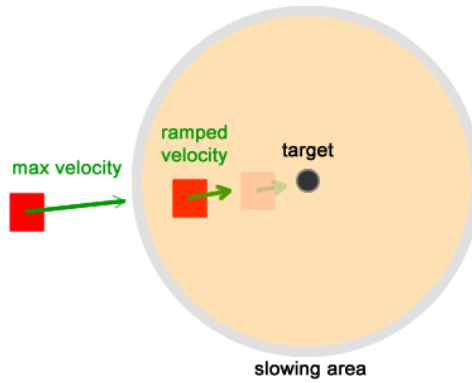


FIGURE 6 – ralentir pour s’arrêter en un point donné

Comportement d’évitement d’obstacles (Avoidance)

L’idée est de générer une force de steering pour s’éloigner des obstacles les plus proches. On ne s’intéressera qu’à l’obstacle le plus proche situé devant l’entité.

On raisonne ici en 2d : les placements et déplacements se font dans le plan horizontal ou dans un plan co-planaire à celui-ci.

- **Détection de collision** : on utilise un "palpeur" virtuel P' que l’entité place devant lui dans le sens du déplacement.

$$P' = P + k\vec{v}_c$$

avec

- P : la position de l’entité
- $\vec{v}_c$: la vitesse courante de l’entité
- C : le centre du volume englobant
- r : le rayon du volume englobant

Il y a possibilité de collision quand $|P'C| < r$ (le palpeur est dans le volume englobant l’obstacle)).

- **Calcul de la force d’évitement** En cas de possibilité de collision on applique à l’entité une force $\frac{k'}{|CP'|}C\vec{P}'$ avec $k' > 0$

Combinaison de comportements

Il est possible d’envisager des comportements complexes résultant de la combinaison de plusieurs comportements élémentaires. Par exemple suivre une trajectoire (comportement C1), tout en longeant un mur (comportement C2) et en évitant des obstacles (comportement C3).

D’après le principe de steering à chaque comportement élémentaire C_i évoqué ci-dessus correspond une force de steering $\vec{F}_i$ appliquée à l’entité contrôlée.

6.4 Exercice(s)

Exercice

Proposez un composant Trajectoire qui permet à une entité, par exemple le guide que vous avez déjà mis en oeuvre de passer par $N - 1$ points. Proposez des options qui permettent de boucler sur la trajectoire ou de s'arrêter sur le dernier point spécifié. L'entité ainsi contrôlée est toujours orientée de façon à regarder dans le sens du déplacement et ne pas donner l'impression de se déplacer en crabe.

Attention : vérifiez dans quelle mesure la classe dont l'entité concernée est une instance est adaptée aux composants utilisés.

Exercice optionnel

Donnez la possibilité d'un contrôle de la caméra virtuelle par steering.

7 Simulation de foule

7.1 Objectif

On veut simuler un système complexe tel qu'une foule. Le comportement d'un tel système ne peut pas se limiter à la somme de comportements individuels.

Ce système est caractérisé par les points suivants :

- le comportement de chaque élément dépend de celui des autres
- l'environnement n'est pas déterminé a priori
- les règles qui définissent le comportement des membres du système sont identiques (tout en acceptant une variabilité sur quelques paramètres)

7.2 Principe

Reynolds, dans son article de 87 propose trois règles permettant de faire vivre un groupe d'agents (une foule ?) Ces règles permettent à la fois d'assurer la cohésion du groupe tout en évitant les collisions entre ses différents membres.

Pour rester dans le même esprit que ce qui a été vu précédemment : chaque règle qui régit le comportement d'un élément du système conduit à l'application d'une force à cet élément.

Trois règles sont appliquées :

- une règle de séparation $\vec{F}_s$: elle permet d'éviter les collisions entre les différents éléments du système
- une règle de cohésion $\vec{F}_c$ qui permet aux différents éléments du système de rester groupés
- une règle d'alignement $\vec{F}_a$ qui permet aux différents éléments du système de se déplacer dans la même direction avec la même vitesse.

On applique à chaque objet une force :

$$\vec{F}_b = ks\vec{F}_s + kc\vec{F}_c + ka\vec{F}_a$$

Règle de séparation : $\vec{F}_s$

Cette règle permet d'éviter les collisions entre éléments du système considéré. Elle consiste à appliquer sur l'élément O_i une force de répulsion engendrée par les éléments O_j du voisinage de O_i .

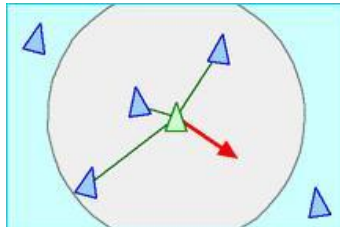


FIGURE 7 – règle de séparation

P_i (resp. P_j) représente O_i (resp. O_j). On peut exprimer cette force de la façon suivante :

$$\vec{F}_s = \sum_{O_j \in \text{voisinage}(O_i)} \frac{1}{|\vec{P_j P_i}|^2} P_j \vec{P_i}$$

En première approximation le voisinage de O_i est une sphère de rayon r_s centrée sur P_i .

Règle de cohésion : $\vec{F}_c$

Cette règle permet d'éviter que les différents éléments d'un groupe ne s'éloignent trop conduisant à la disparition de ce groupe.

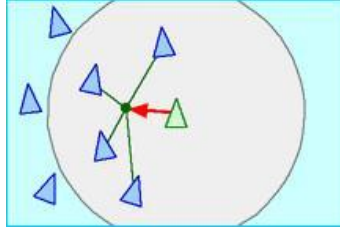


FIGURE 8 – Règle de cohésion

On peut exprimer cette force de la façon suivante :

$$\vec{F}_c = seek(O_i, G)$$

avec

- $seek(O, P)$: la force de steering qui permet de faire passer l'objet O par le point P
- $G : G = \frac{1}{|\text{voisinage}(O_i)|} \sum_{O_j \in \text{voisinage}(O_i)} P_j$ (le centre de gravité des voisins de O_i)

Règle d'alignement : $\vec{F}_a$

Dans un groupe, un moyen de garder la structure de groupe tout en évitant les collisions consiste à imposer le même mouvement de translation à tous les éléments d'un groupe. On cherche donc à imposer la même vitesse $\vec{v}_m$ à tous les éléments d'un même groupe (en l'occurrence ici les individus du voisinage de O_i).

$$\vec{v}_m = \frac{1}{|\text{voisinage}(O_i)|} \sum_{O_j \in \text{voisinage}(O_i)} \vec{v}_j$$

La force $\vec{F}_a$ correspond à la force à appliquer pour passer de la vitesse $\vec{v}_i$ de l'objet O_i à la vitesse $\vec{v}_m$.

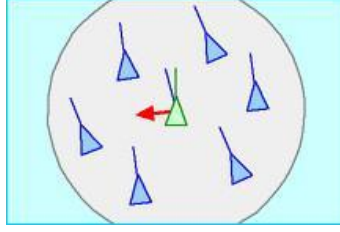


FIGURE 9 – règle d'alignement

$$\vec{F}_a = \vec{v}_m - \vec{v}_i$$

Combinaison des règles

Comme indiqué plus haut les forces résultantes de ces trois règles élémentaires sont combinées linéairement :

$$\vec{F}_b = ks\vec{F}_s + kc\vec{F}_c + ka\vec{F}_a$$

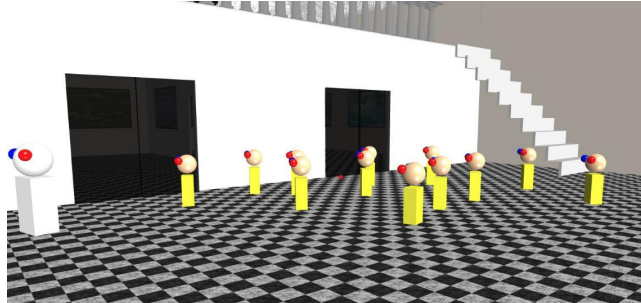
avec $ks + kc + ka = 1$

L'affectation des coefficients k_s , k_a , k_c peut être soit globale (la même pour toutes les entités du groupe) soit locale (choix individuel pour chaque entité du groupe).

Toujours est-il que le choix de ces coefficients a une conséquence sur le comportement émergent de groupe : en favorisant un coefficient au détriment des autres on favorise un aspect comportemental plutôt qu'un autre.

Par exemple si ks est important il y aura moins de collisions entre les entités mais l'aspect de groupe va s'atténuer.

7.3 Exercices



Exercice

Créez un groupe de 10 entités qui représentent les visiteurs d'un musée. Appliquez leur les règles décrites ci-dessus. Faites en sorte que ces visiteurs non seulement aient un comportement de groupe "crédible" mais également qu'ils restent confinés dans une région (le hall de votre musée par exemple). Faites en sorte que chaque membre du groupe s'oriente dans le sens de son déplacement.

Exercice

Faites en sorte que les membres du groupe s'écartent de la caméra virtuelle quand celle-ci est trop proche d'eux.

Exercice

On ajoute au groupe de visiteurs une entité distincte : un guide. Celui-ci se déplace dans une région donnée (par exemple le hall de votre projet) selon une trajectoire qui lui est propre (définie a priori ou calculée au fur et à mesure de la simulation). Faites en sorte que le groupe de visiteurs suivent (a) le guide (b) un point situé derrière le guide. Quelle différence constatez vous.

Exercice

Faites en sorte que le groupe, suivant le guide, s'arrête régulièrement devant un tableau et se mette à le regarder puis, après quelques secondes se remet en route vers un autre tableau.

Exercice optionnel

Faites en sorte que la caméra virtuelle puisse

1. être contrôlée totalement par un programme de flocking comme tous les autres visiteurs
2. être contrôlée partiellement comme tous les autres visiteurs : l'utilisateur contrôle la direction du regard mais pas la position

Exercice optionnel

Faites en sorte que le groupe, suivant le guide, puisse passer du hall à une des salles du musée et ceci de façon aussi crédible que possible.

8 Annexe : Vector3

```
— u.set(x,y,z) : u := (x,y,z)
— u.addInPlace(Vector3 v) : u := u + v
— u.subtractInPlace(v) : u := u - v
— u.subtractToRef(v, w) : w := u - v
— u.scaleAndAddToRef(k,v) : v := v + k*u
— d = BABYLON.Vector3.Distance(u,v)
```